Group 28 Progress Report: Spoiler Review Classifier

Sky Quan, Aidan Lao, Hanna Arendt
{quans6,laoa6,arendth}@mcmaster.ca

1 Introduction

Many people browse reviews before starting a book. There are popular platforms like Goodreads to facilitate this, but reviews often contain major spoilers. This can ruin the experience of reading the book and upset readers that just want to hear a high-level overview or opinions. Goodreads lets reviewers hide this content behind spoiler tags, but it is based on trust, since humans decide whether to include the spoiler tags or not. Because of this, readers can have a book spoiled before they ever open it even if it doesn't have a spoiler tag. So, creating a spoiler detector keeps readers safe by checking for untagged spoilers before they are posted, letting readers safely filter through reviews that don't have spoilers. This would make review platforms safer for readers to get a high-level overview of a book without worrying about being spoiled.

2 Related Work

There are plenty of studies regarding spoiler detection through machine learning from the last decade. In 2013, Boyd-Graber et al. collected spoilers from social media and set a baseline with lexical features. They then showed that metadata-based features improved performance substantially. Another study done by Jeon et al. targeted TV program tweets, as previous models only performed well on larger bodies of texts (2016). By building a SVM-based prediction model, utilizing four identified features, they created a model that achieved greater precision and comparable recall.

There are also more recent work that uses neural networks. Whereas the earlier studies were based on hand-crafted domain-specific features, one study used a deep-neural spoiler detection model that used a genre-aware detection mechanism to achieve even higher generalizability (Chang et al., 2018). Likewise, a study from 2025 focusing on spoilers in movie reviews from platforms such as IMDb showed that spoiler rates differ by genre and that some users post spoilers more often, and built a model that combines genre information with graph-based user bias (Zhang et al.).

The work most similiar to ours is the study our dataset is retrieved from. They developed an end-to-end neural network architecture to detect spoilers based on findings from analyzing their data, such that spoiler language tends to be book-specific (Wan et al., 2019).

3 Dataset

The source dataset is the Goodreads Spoilers Dataset, originally collected and released by McAuley Lab at UC San Diego, and reposted on Kaggle by Ahmad Mujtaba.

- Original source: https://cseweb.ucsd.edu/~jmcauley/datasets/goodreads.html
- Kaggle link: https://www.kaggle.com/datasets/pypiahmad/goodreads-book-reviews

The data was collected in late 2017 from goodreads.com by scraping users' public Goodreads shelves. Given the dataset size of 1,378,033 reviews (of 25,475 books by 18,892 users), we use a smaller, randomly sampled subset of data composed of 10,000.

To obtain this smaller and randomized subset, we used stratified reservoir sampling. This was used for two main reasons. First, to give every datapoint a randomized chance to appear in our smaller subset. Secondly, to combat the class imbalance within the source dataset. Since the dataset contains many more reviews without spoilers compared to those with, randomly sampling datapoints doesn't give the model enough spoilers to train on. So, stratified reservoir sampling allows

us to also sample a predefined number of spoilers and non-spoilers. With the parameters we set, we create a dataset with 10,000 reviews where 20% of the reviews contain spoilers and 80% of the reviews do not, regardless of class imbalance.

The source dataset does not label each review as containing or spoiler or not. However, every spoiler within the dataset contains the text `(view spoiler)[...(hide spoiler)]`, where `...` represents the specific spoiler to each book. So, if a review contains the text `(view spoiler)` or `(hide spoiler)`, it was labelled as 1, contains a spoiler, and if it did not contain the text, it was labelled as 0, contains no spoiler. The labels were placed under the newly created column has_spoiler.

The dataset is then cleaned by removing the unnecessary columns ['userid', 'rating', 'date_added', 'date_updated', 'read_at', 'started_at', 'n_votes', 'n_comments'] while keeping the necessary columns ['review_id', 'book_id', 'review_text', 'has_spoiler']. The spoiler tags `(view spoiler)[` and `(hide spoiler)]` are then removed from review_text. Finally, the dataset is split into training, validation, and testing data by book, with 80%, 10%, and 10% of the books respectively, so that all reviews of a given book appear in only one of the three sets (see Section 5).

Two example datapoints, with review_text shortened for readability:

```
{'book_id': '16981',
 'review_id':
   'a5d2c3628987712d0e05c4f90798eb67',
 'review_text': 'Recommended by Don Katz.
   Avail for free in December...',
 'has_spoiler': 0}

{'book_id': '28684704',
 'review_id':
   '2ede853b14dc4583f96cf5d120af636f',
 'review_text': '... can't be discussed
   without spoilers ... Daniela says, ...',
 'has_spoiler': 1}
```

4 Features

Each review is represented as a bag-of-words vector weighted by TF-IDF (term frequency–inverse document frequency) (Salton and Buckley, 1988), built with scikit-learn's `TfidfVectorizer` (Pedregosa et al., 2011). A term gets a high weight when it appears often in a review but rarely across the training set, so common filler words count for little and more specific words such as *killed* or *ending* count for more. We use the following settings:

- **Unigrams and bigrams** (`ngram_range=(1,2)`). Many spoiler cues are short phrases rather than single words, e.g. *the end* or *she dies*.
- **Lowercasing**, so *Ending* and *ending* are the same feature.
- **No stop-word removal.** Pronouns such as *he* and *she* turned out to be among the strongest spoiler features, since spoilers tend to retell what characters did. Removing them would throw this signal away.
- **`min_df=2`** drops terms that appear in only one review. These are mostly typos and one-off names that cannot generalize.
- **`max_df=0.9`** drops terms that appear in more than 90% of reviews.
- **Sublinear term frequency** $(1 + \log \mathrm{tf})$, so a word repeated many times in a long review does not dominate the vector.
- **Vocabulary capped at 50,000 terms**, keeping the most frequent ones.

The vectorizer is fit on the training set only and then applied to the validation and test sets. This way, words that only occur in held-out books do not enter the vocabulary, which matches how the model would be used on new books.

We do not do any further feature selection, augmentation or learned embeddings yet. Masking book-specific words (character and place names) is planned as the next feature step (Section 7).

5 Implementation

All code is written in Python with pandas and scikit-learn and is in `models/train_logreg.py`.

Splitting by book. We split the 10,000 reviews into training, validation and test sets with `GroupShuffleSplit`, using `book_id` as the group. First 80% of the books go to training, then the remaining books are split evenly between validation and test. Every review of a given book therefore appears in exactly one set, and the script checks that no book is shared between sets. This tests whether the model can find spoilers in reviews of books it has never seen, rather than memorizing plot details of specific books. Because books have different numbers of reviews, the split is close to, but not exactly, 80/10/10 by review count. Table 1 shows the result. The spoiler share stays close to 20% in every set.

Split	Reviews	Books	Spoiler %
Train	8,015	5,108	20.1
Validation	1,014	639	20.3
Test	971	639	18.8

Table 1: Book-based split. No book appears in more than one set.

Weights	C	P	R	F1
Equal	0.1	0.00	0.00	0.00
Equal	1	0.61	0.17	0.27
Equal	10	0.51	0.28	0.36
Balanced	0.1	0.36	0.69	0.48
Balanced	1	0.41	0.65	**0.50**
Balanced	10	0.46	0.50	0.47

Table 2: Validation precision (P), recall (R) and F1 on the spoiler class for a subset of the tuning grid.

Majority-class baseline. The baseline always predicts the most common class in the training set, which is "no spoiler". It reaches 79.7% accuracy on the validation set while finding no spoilers at all, so its spoiler-class F1 is 0. This shows why accuracy alone is misleading for this task.

Logistic regression. Our main model is L2-regularized logistic regression on the TF-IDF features. It models the probability that a review contains a spoiler as $p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ and is trained by minimizing the regularized cross-entropy (log) loss

$$\frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \alpha_{y_i} \, \ell\big(y_i, \sigma(\mathbf{w}^\top \mathbf{x}_i + b)\big), \quad (1)$$

where ℓ is the log loss, C controls the strength of regularization (smaller C means stronger regularization), and α_y is an optional per-class weight. The model is optimized with scikit-learn's `liblinear` solver, a coordinate-descent method suited to sparse, high-dimensional text features. A review is predicted as a spoiler when $p(y = 1 \mid \mathbf{x}) \geq 0.5$.

Tuning. We tuned two hyperparameters on the validation set: $C \in \{0.01, 0.1, 1, 10, 100\}$ and the class weights, either equal or *balanced*. Balanced weights scale each class inversely to its frequency, so a missed spoiler costs about four times as much as a false alarm. We picked the setting with the highest spoiler-class F1 on validation (Table 2). The test set was not used for any of these choices.

Class weighting mattered far more than C. With equal weights and strong regularization, the model collapses to the baseline and predicts no spoilers. Even at its best, it finds fewer than a third of the spoilers. Balanced weights roughly double recall at the cost of some precision, and every balanced setting beats every equal-weight setting on F1. The selected model uses balanced weights and $C = 1$, reaching a validation F1 of 0.50 (precision 0.41, recall 0.65), against 0 for the baseline.

6 Results and Evaluation

The model is evaluated through precision, recall and F1 on the spoiler class, which is treated as the positive class. We compare against a majority-class baseline, as earlier mentioned in section 5. Table 3 shows the results. The balanced logistic regression reaches a test F1 of 0.506, with precision of 0.408 and recall of 0.667. This is far above the majority baseline (F1 of 0), and the test score is nearly identical to the validation F1 of 0.502, so the tuning did not overfit the validation set.

Set	Model	Acc	P	R	F1
Validation	Majority	0.797	0.000	0.000	0.000
Validation	LogReg	0.740	0.410	0.646	0.502
Test	Majority	0.812	0.000	0.000	0.000
Test	LogReg	0.755	0.408	0.667	0.506

Table 3: Spoiler-class results.

The validation confusion matrix (Figure 1) shows how the model behaves on the data we tuned on: of the 324 reviews flagged as spoilers, 191 are false alarms, and 73 of the 206 real spoilers are missed. The test confusion matrix (Figure 2) shows the same pattern on unseen reviews. Of the 299 reviews the model flags as spoilers, 177 (59%) are false alarms, while 61 of the 183 real spoilers (33%) are missed. This is the precision-for-recall trade-off created by the balanced class weights. In practice, a reader using this filter would lose many harmless reviews, though most spoilers would still be caught.

Inspecting the learned weights shows that the strongest spoiler features are plot-retelling words: *killed*, *died*, *ending*, *the end*, *scene*, *cheating*, and the pronouns *he* and *she*. The strongest non-spoiler features are about the review itself rather than the plot: *read*, *review*, *blog*, *publisher*, *http*. Interestingly, *spoiler alert* points toward non-spoiler, likely because reviewers use the phrase to promise a spoiler-free review.

7 Feedback and Plans

On our proposal, our TA, Abdelaziz Serour, told us to remove the Goodreads spoiler markers after

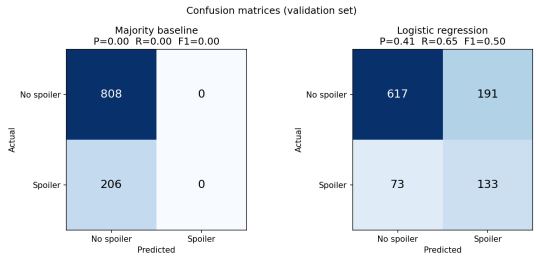


Figure 1: validation matrix

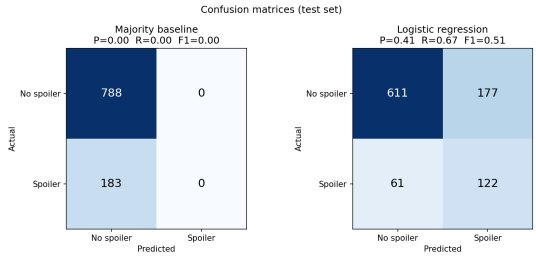


Figure 2: test matrix

Spoiler term	Weight	Non-spoiler term	Weight
he	2.718	read	−1.702
was	2.522	review	−1.450
that	2.311	spoiler alert	−1.154
also	2.290	alert	−1.133
ending	2.120	http	−1.099
killed	2.081	com	−1.079

Table 4: Weights for spoiler words and non-spoiler words

creating the labels, and to check the class distribution before fixing a sample size. Based on that, we made the decision to do the stratified 10,000-review sample with 20% spoilers. He advised building TF-IDF and logistic regression first, with a strong evaluation pipeline, before adding NER masking or a neural network, and to use precision, recall, F1 and confusion matrices instead of accuracy. We followed this order, and the results above come from that pipeline.

Our next step is name and place masking. Since we split by book, character and place names cannot transfer between books, but they may still push predictions toward spoiler. We will use spaCy NER to replace people and places with placeholders such as [PERSON] and [LOCATION]. Futhermore, we will investigate on if we will also mask terms that appear almost only in one book, and if so do a Python script for that purpose. We will then rerun the same logistic regression with the same split, tuning and evaluation function. The only change is the input, so any difference in F1 can be credited to masking.

After masking, we will train neural network models in PyTorch, building on the neural network and convolutional neural network material from the course. We will do this in two steps, so that each change can be measured on its own:

- **Feed-forward network on TF-IDF.** First, we will train a small feed-forward network (one or two hidden layers with ReLU activations and dropout) on the same TF-IDF features as the logistic regression. Since the input is unchanged, this shows whether a nonlinear model alone improves over logistic regression.
- **CNN over word embeddings.** Next, we will train a convolutional neural network that reads the review as a sequence of word embeddings, with filters over windows of a few words. Unlike bag-of-words, this lets the model use word order and short phrases, such as *"she dies at the end"*. Reviews will be cut or padded to a fixed length, and we will check that this does not hide spoilers that appear late in long reviews, since Wan et al. (2019) found that spoilers tend to appear in the latter part of reviews.

Both networks will output a spoiler probability through a sigmoid and be trained with binary cross-entropy loss using the Adam optimizer. To handle the class imbalance in the same way as our balanced logistic regression, spoiler examples will be weighted about four times higher in the loss. We will tune the learning rate, hidden size and dropout on the validation set, and use early stopping on validation F1 to avoid overfitting. Each network will use the same book-based split, masked and unmasked inputs, and the same evaluation function as logistic regression, so all models are compared on spoiler-class precision, recall, F1 and confusion matrices.

Team Contributions

Sky Quan: Handled everything involving the dataset. Loaded the source dataset and completed all the preprocessing: cleaning, labelling, and spliting the dataset as required. Wrote the Introduction and Dataset section.

Aidan Lao: Built and trained the TF-IDF logistic regression model, including the book-based train/validation/test split and hyperparameter tuning. Wrote the evaluation script, which computes

the metrics and generates the confusion matrices. Wrote the Features and Implementation sections.

Hanna Arendt: Ran the final evaluation of the model on the validation and test sets, and wrote the Related Work, Results and Evaluation, and Feedback and Plans sections.

References

Jordan Boyd-Graber, Kimberly Glasgow, and Jackie Sauter Zajac. 2013. Spoiler alert: Machine learning approaches to detect social media posts with revelatory information. *Proceedings of the American Society for Information Science and Technology*, 50(1):1–9.

Buru Chang, Hyunjae Kim, Raehyun Kim, Deahan Kim, and Jaewoo Kang. 2018. A deep neural spoiler detection model using a genre-aware attention mechanism. In *Advances in Knowledge Discovery and Data Mining*, pages 183–195, Cham. Springer International Publishing.

Sungho Jeon, Sungchul Kim, and Hwanjo Yu. 2016. Spoiler detection in tv program tweets. *Information Sciences*, 329:220–235. Special issue on Discovery Science.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Gerard Salton and Christopher Buckley. 1988. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523.

Mengting Wan, Rishabh Misra, Ndapa Nakashole, and Julian McAuley. 2019. Fine-grained spoiler detection from large-scale review corpora. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2605–2610, Florence, Italy. Association for Computational Linguistics.

Haokai Zhang, Shengtao Zhang, Zijian Cai, Heng Wang, Ruixuan Zhu, Zinan Zeng, and Minnan Luo. 2026. Unveiling the hidden: Movie genre and user bias in spoiler detection. In *Machine Learning and Knowledge Discovery in Databases. Research Track*, pages 19–36, Cham. Springer Nature Switzerland.